UCAD — DISI • MASTER 1 / LICENCE

Architecture des applications

Du client/serveur aux microservices



Administration systèmes • Réseaux • Infrastructure • Exploitation

Pourquoi parler d’architecture applicative

Une plateforme universitaire doit être accessible par plusieurs milliers d’utilisateurs

Où exécuter l’application ?

Serveur physique, VM, cloud, environnement hybride.

Comment séparer les rôles ?

Interface, traitement applicatif, données, sécurité.

Comment stocker les données ?

Base de données, fichiers, sauvegardes, stockage centralisé.

Comment répartir la charge ?

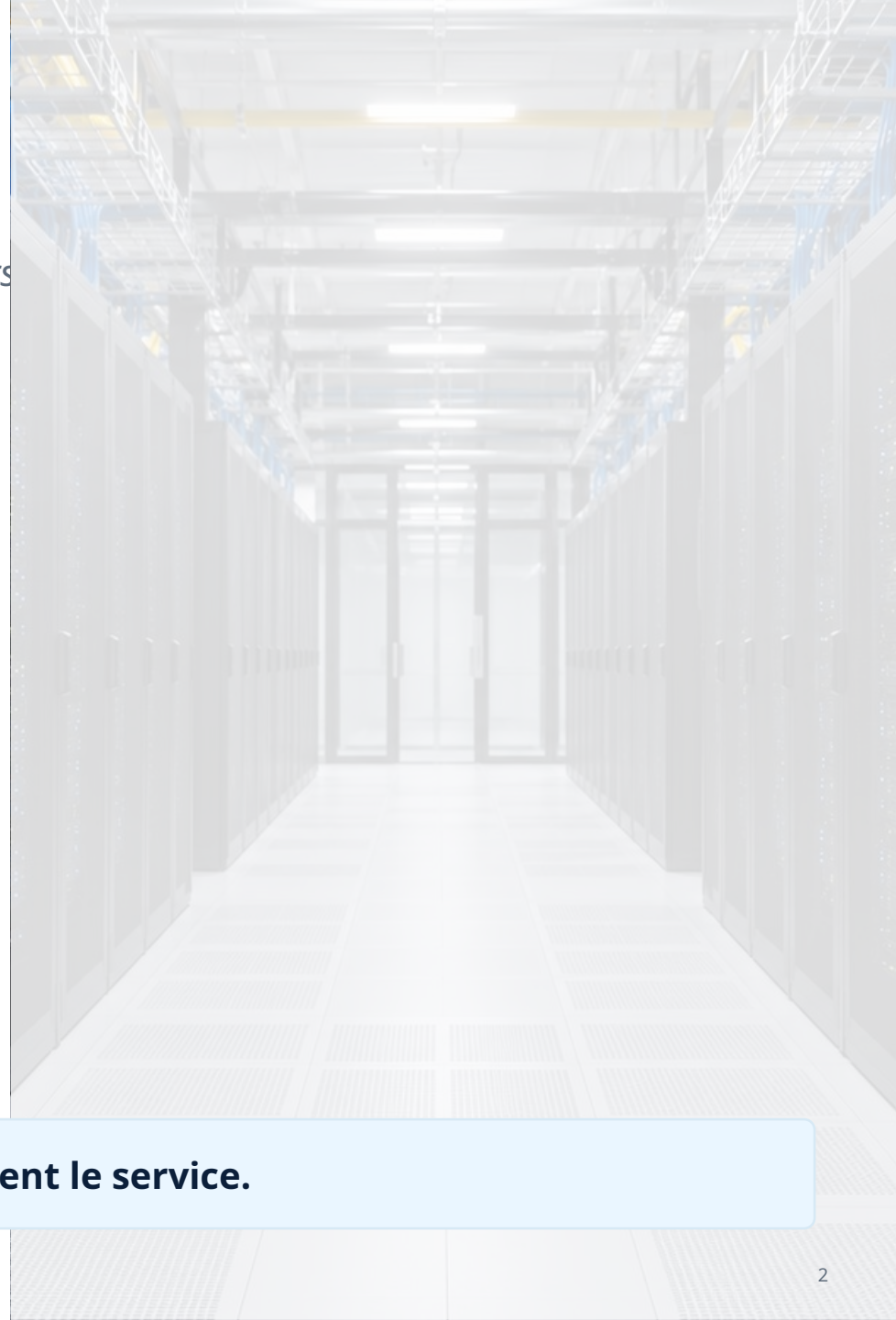
Plusieurs serveurs, load balancer, redondance.

Comment sécuriser les échanges ?

HTTPS, pare-feu, authentification, segmentation.

Comment garantir la disponibilité ?

Supervision, sauvegarde, tolérance aux pannes.



L’architecture applicative définit l’organisation des composants qui rendent le service.

Principe client / serveur

Un client demande un service ; un serveur le fournit.

Client

Initie la communication.
Envoie une requête.
Consomme un service.

Serveur

Écoute sur un port.
Traite la demande.
Renvoie une réponse.





Exemple simple : un navigateur web contacte un serveur pour obtenir une page.

Communication client / serveur

La communication se fait par requête, traitement et réponse.

1. Requête

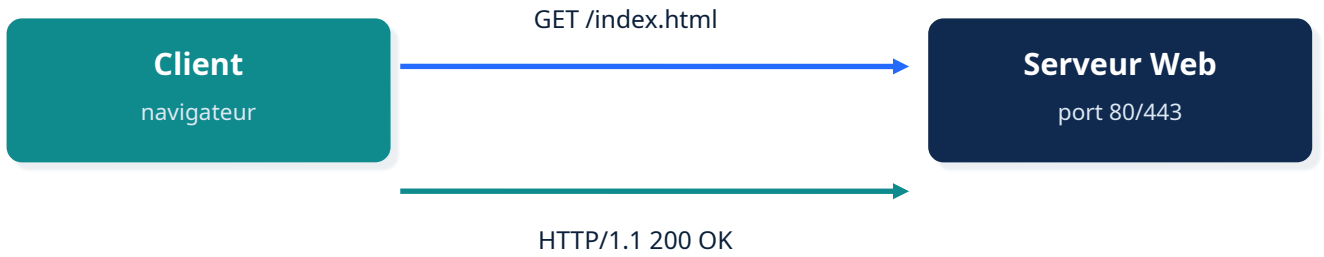
Le client envoie une demande : page, fichier, action.

2. Traitement

Le serveur interprète la demande et mobilise ses services.

3. Réponse

Le serveur retourne une page, des données ou une erreur.



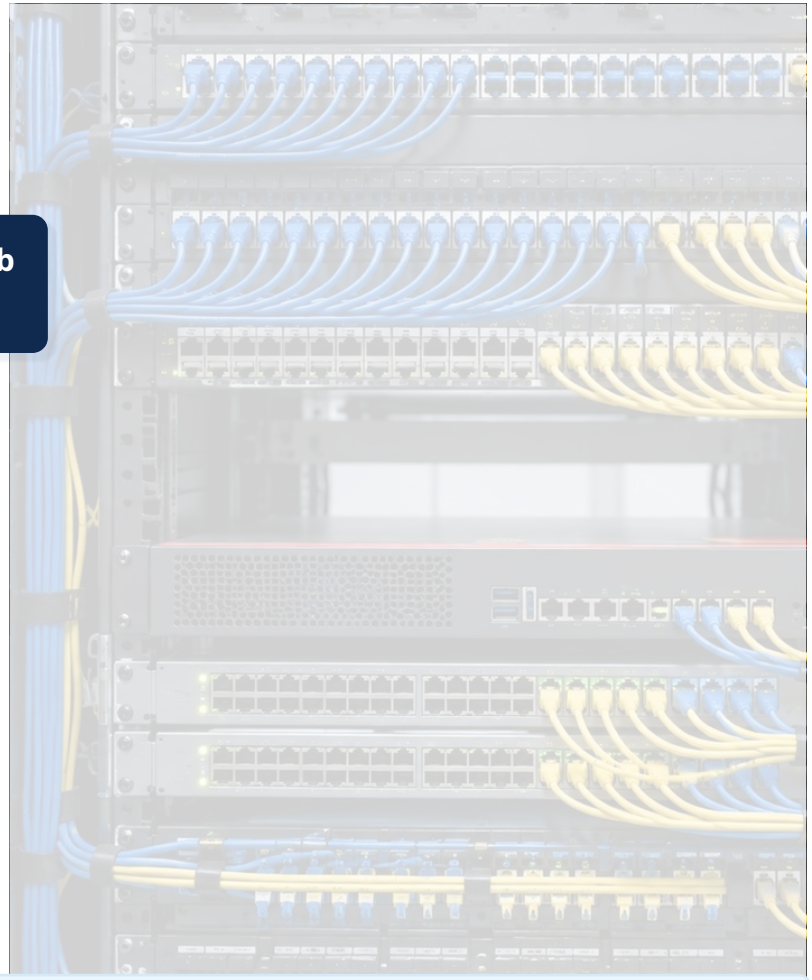
Exemple de flux

```
Client → HTTP/HTTPS → Serveur Web
Protocole : HTTP ou HTTPS
Adresse IP : serveur cible
Port : 80 pour HTTP, 443 pour HTTPS
```

Exemple réel client / serveur

Un étudiant ouvre le portail universitaire depuis Internet.





Lecture technique du parcours

Le DNS traduit le nom du portail en adresse IP.
Le pare-feu contrôle l'accès entrant.
Le serveur web fournit la page demandée.

L'utilisateur voit une page ; l'architecte voit un chemin réseau complet.

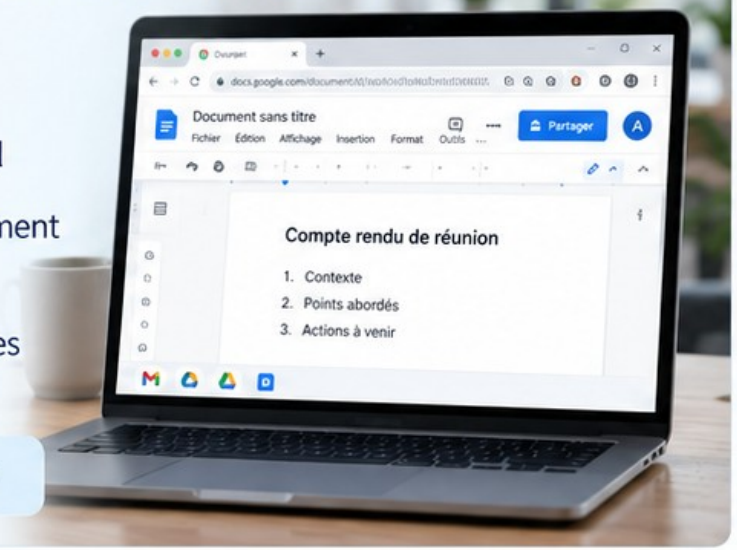
Client léger et client lourd

Le type de client influence le déploiement, la maintenance et la sécurité.

Client léger

UNE APPLICATION ACCESSIBLE DEPUIS UN NAVIGATEUR

- Navigateur web
- Peu de traitement local
- Application principalement côté serveur
- Mises à jour centralisées

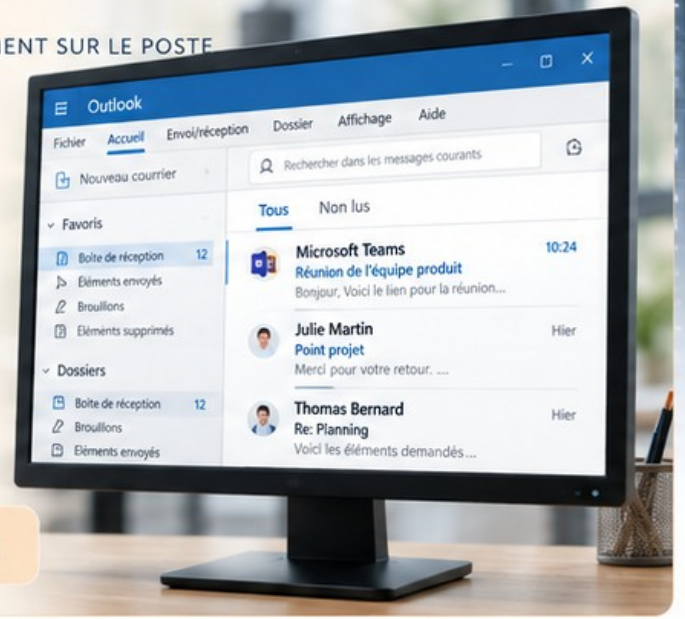


S'exécute dans le navigateur

Client lourd

UN LOGICIEL INSTALLÉ LOCALEMENT SUR LE POSTE

- Logiciel installé sur le poste
- Traitements locaux plus importants
- Dépendance au système client
- Mises à jour poste par poste



Installé sur le poste (application locale)

Comparaison synthétique

Déploiement

Client léger	Simple et centralisé
Client lourd	Plus complexe et local

Maintenance

Client léger	Facile et centralisée
Client lourd	Plus lourde, poste par poste

Sécurité

Client léger	Surface réduite, plus maîtrisée
Client lourd	Plus exposé (dépend du poste)

Usage typique

Client léger	Applications web modernes
Client lourd	Environnements métier spécifiques

Le client léger est fréquent dans les applications web modernes ; le client lourd existe encore dans certains environnements métier.

PARTIE 2

Architecture 2-tiers

Client et serveur de données

07

Définition du 2-tiers

Le client communique directement avec un serveur ou une base de données.



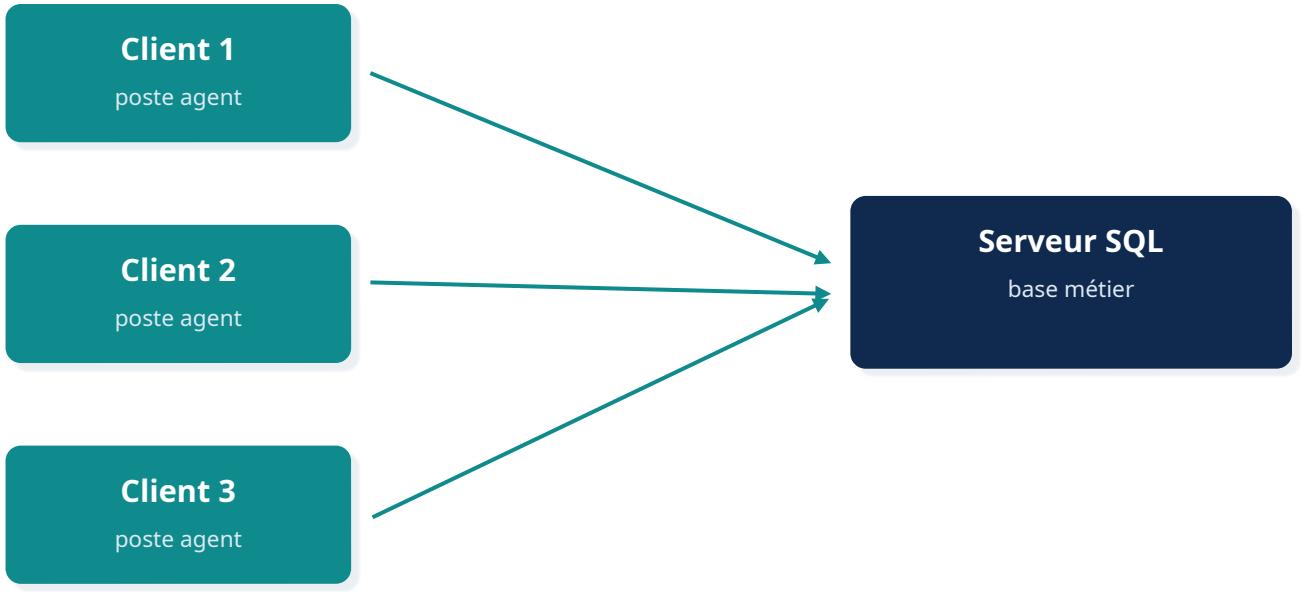
Idée centrale

Deux niveaux principaux : poste client et serveur. Le client porte souvent une partie importante du traitement.



Exemple 2-tiers

Une application de gestion installée sur 20 postes accède directement au serveur SQL





Simple à comprendre, mais la base de données devient rapidement un composant très exposé.

Avantages du 2-tiers

Un modèle simple, rapide à déployer et facile à comprendre.

Simplicité

Peu de composants et flux faciles à identifier.

Déploiement rapide

Adapté aux petits services ou aux environnements internes.

Coût initial réduit

Infrastructure légère et peu de serveurs.

Administration simple

Moins d'éléments à superviser au départ.



Le 2-tiers est pertinent quand le besoin est limité, stable et bien maîtrisé.

Limites du 2-tiers

Les limites apparaissent lorsque le nombre d'utilisateurs ou les exigences augmentent

Dépendance forte

Le client dépend directement du serveur de données.

Montée en charge limitée

Le serveur devient vite un point de concentration.

Mises à jour difficiles

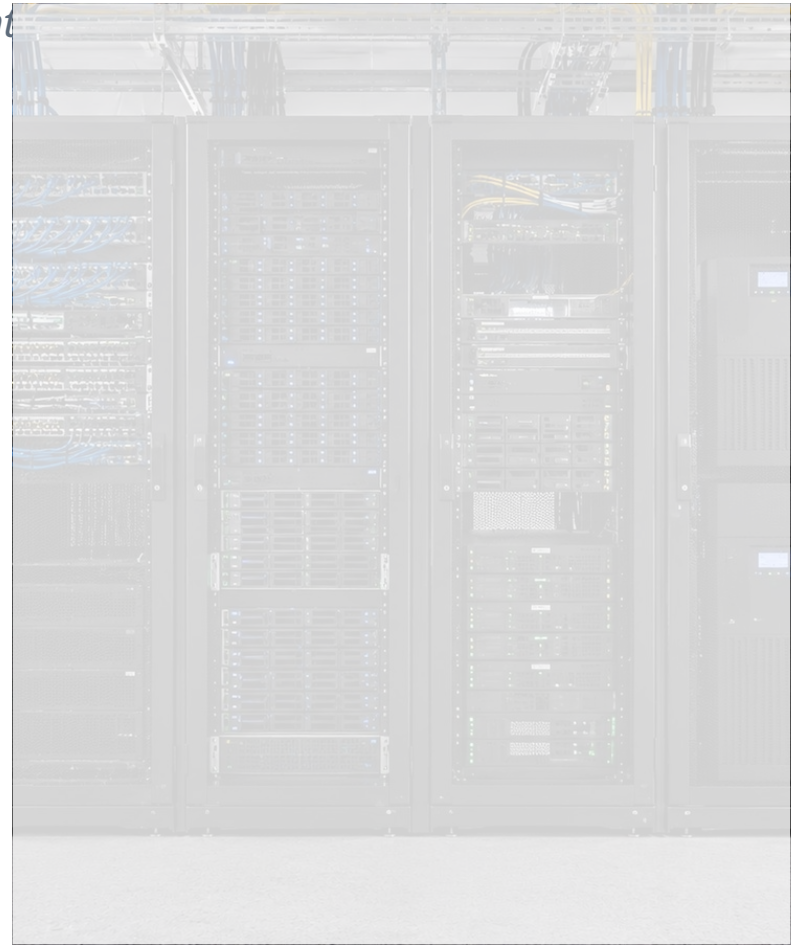
Chaque poste client peut devoir être maintenu.

Exposition de la base

La base est plus directement accessible par les clients.

Quand éviter ?

Applications exposées sur Internet, grand nombre d'utilisateurs, exigences fortes de sécurité ou de disponibilité.



PARTIE 3

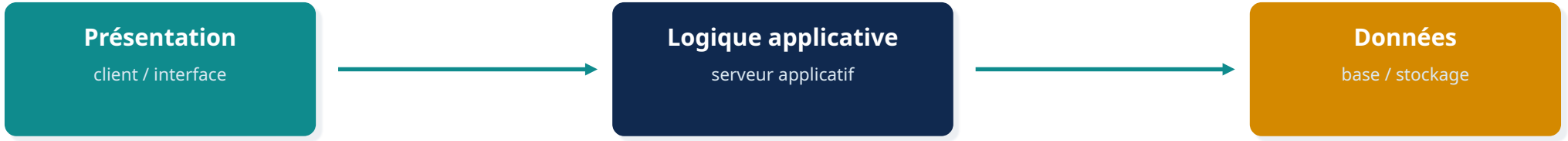
Architecture 3-tiers

Présentation, application et données

12

Définition du 3-tiers

Le service est organisé en trois niveaux principaux.



Tier 1

Navigateur, application mobile ou interface utilisateur.

Tier 2

Règles métier, traitements, sessions, API.

Tier 3

Base de données, fichiers, stockage persistant.

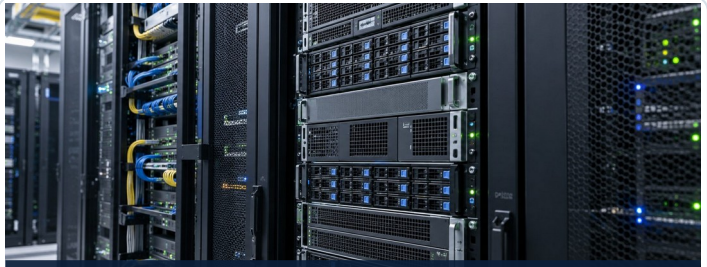
Les trois tiers

Chaque tier répond à une responsabilité technique précise.



Tier 1 — Présentation

Interface utilisée par l'utilisateur : navigateur, application mobile, poste client.



Tier 2 — Traitement

Serveur applicatif : règles, sessions, calculs, appels aux services.



Tier 3 — Données

Base de données et stockage : conservation durable des informations.

Séparer les tiers facilite la sécurité, la maintenance et la montée en charge.

Exemple universitaire en 3-tiers

L'étudiant utilise le portail, le serveur applicatif traite, la base fournit les données.



Requête du client

```
GET /api/notes HTTP/1.1
Host: portail.universite.sn
Authorization: Bearer ...
```

Traitement côté serveur

```
SELECT ue, note FROM notes
WHERE etudiant_id = 123;
```

Le client ne contacte pas directement la base : il passe par le serveur applicatif.

Intérêt du 3-tiers

Le 3-tiers rend l'architecture plus structurée et plus exploitable.

Séparation des responsabilités

Chaque tier a un rôle précis.

Sécurité améliorée

La base n'est pas directement exposée.

Évolutivité

On peut renforcer le tier le plus sollicité.

Maintenance facilitée

L'application et les données évoluent séparément.

Répartition possible

Les composants peuvent être placés sur différentes VM ou serveurs.



Le 3-tiers est un modèle de base des applications Web d'entreprise.

2-tiers vs 3-tiers

Deux modèles différents par leur séparation des rôles et leur capacité d'évolution.

2-tiers	3-tiers
Client ↔️ serveur ou base	Client ↔️ application ↔️ données
Modèle simple	Modèle plus structuré
Peu de composants	Plusieurs rôles techniques
Montée en charge limitée	Évolutivité plus importante
Séparation faible	Séparation forte des responsabilités

Le 3-tiers convient mieux aux applications Web exposées, évolutives et sensibles.

Attention : tiers ≠ serveurs physiques

Un tier est un rôle logique ; il ne correspond pas toujours à une machine dédiée.

Déploiement physique

3 serveurs réels distincts.

Déploiement virtualisé

3 machines virtuelles sur un cluster.

Déploiement conteneurisé

Plusieurs conteneurs ou services.

Environnement de test

Plusieurs rôles sur une seule machine.



À retenir

Une architecture 3-tiers décrit une organisation logique. Le déploiement réel dépend du budget, de l'environnement, de la charge et des contraintes d'exploitation.

PARTIE 4

Architecture en couches

Organisation logique de l'application

19

Qu’est-ce qu’une architecture en couches ?

Les couches structurent l’application selon des responsabilités internes.

Couche présentation
Interface utilisateur, pages, formulaires.

↓

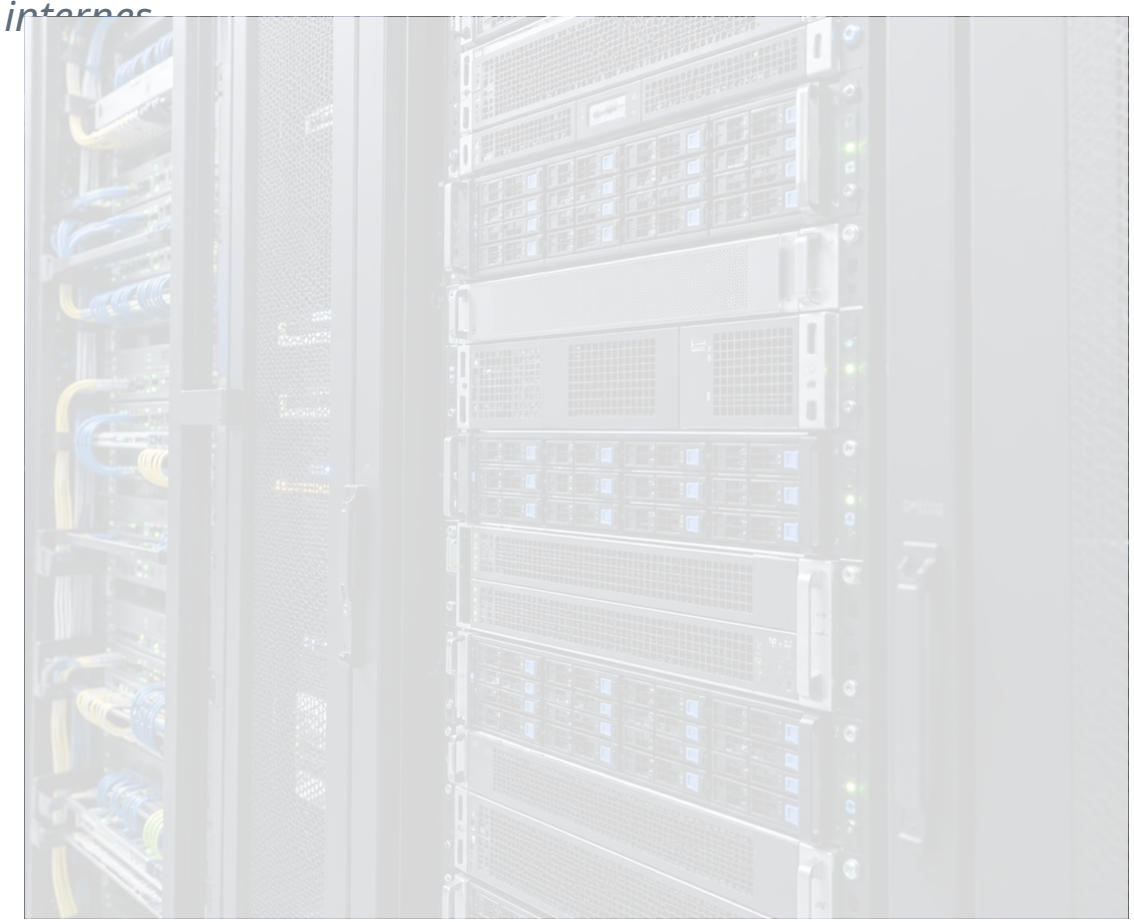
Couche métier
Règles : inscription, validation, paiement, notes.

↓

Couche accès aux données
Requêtes, connexions, lecture/écriture.

↓

Couche données
Base de données et stockage persistant.



Les couches expliquent comment l’application est organisée, pas forcément où elle est hébergée.

Exemple d’architecture en couches

Cas d’une plateforme universitaire.

Présentation

```
Page inscription
Formulaire HTML
Validation utilisateur
```

Services métier

```
ServiceInscription
Valider dossier
Calculer frais
```

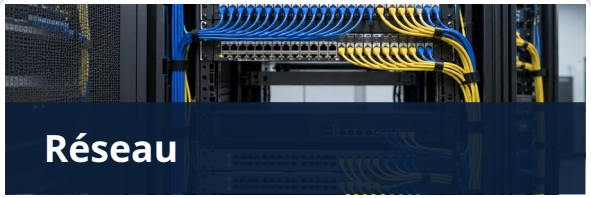
Accès données

```
INSERT INTO
preinscriptions
SELECT notes
```



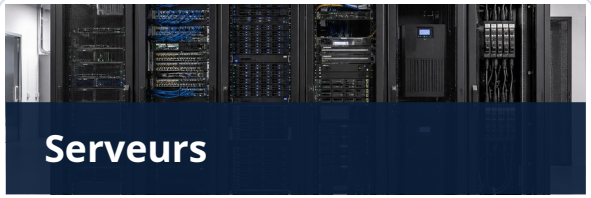
Base

Informations étudiants, formations, paiements.



Réseau

Échanges entre les couches déployées.



Serveurs

Hébergement des composants applicatifs.

Même si les couches sont logiques, elles ont des conséquences sur le déploiement technique.

Tiers vs couches

Deux notions proches, mais pas identiques.

Tiers	Couches
Organisation du déploiement	Organisation logique du logiciel
Où exécuter les composants ?	Quelle responsabilité pour chaque partie ?
Serveur web, serveur applicatif, base	Présentation, métier, accès données
Vision infrastructure	Vision organisation interne
Peut correspondre à des VM ou serveurs	Peut être dans une même application

Une application peut être organisée en couches et déployée en plusieurs tiers.

PARTIE 5

Architecture monolithique

Une application, un ensemble déployable

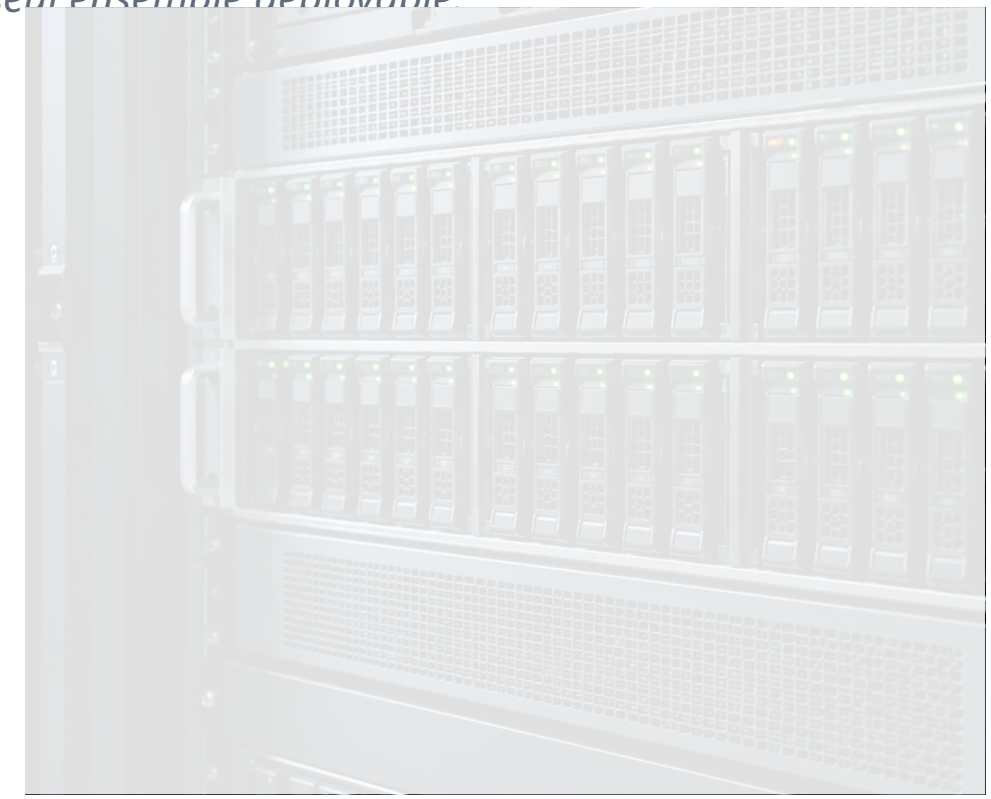
23

Définition du monolithe

Une application regroupe l'essentiel de ses fonctionnalités dans un seul ensemble déployable

Application monolithique

authentification • inscriptions • paiements • notes • attestations • notifications



Monolithique ne signifie pas "mauvais" : c'est souvent un choix simple et cohérent au départ.

Exemple monolithique

La plateforme universitaire contient toutes les fonctions dans une même application.

Inscription

Fonction intégrée au même déploiement.

Paiement

Fonction intégrée au même déploiement.

Notes

Fonction intégrée au même déploiement.

Attestations

Fonction intégrée au même déploiement.

Comptes

Fonction intégrée au même déploiement.

Notifications

Fonction intégrée au même déploiement.

Déploiement unique

une application • une version • une mise à jour

Un seul déploiement peut fournir plusieurs fonctions métier.

Avantages du monolithe

Le monolithe peut être très adapté aux petits et moyens environnements.

Simplicité initiale

Architecture facile à comprendre et à installer.

Déploiement unique

Une version, un paquet, une procédure de mise en production.

Moins de flux internes

Peu de communications réseau entre composants.

Exploitation plus simple

Moins de services à superviser au départ.



Pour une application stable et maîtrisée, le monolithe reste une option crédible.

Limites du monolithe

Les limites apparaissent lorsque l'application grossit ou devient critique.

Croissance plus difficile

La montée en charge se fait souvent sur tout l'ensemble.

Mise à jour globale

Une modification peut imposer un redéploiement complet.

Impact de panne

Une erreur peut affecter plusieurs fonctions à la fois.

Dépendances fortes

Les modules deviennent difficiles à isoler.

Scalabilité moins ciblée

Impossible de renforcer uniquement un service précis.

Plus l'application devient stratégique, plus l'architecture doit prévoir la séparation, la disponibilité et l'évolution.

PARTIE 6

Introduction aux microservices

Découper l'application en services autonomes

28

Pourquoi découper une application ?

Quand l'application grossit, tous les modules n'ont plus les mêmes contraintes.

Plus d'utilisateurs

Certaines fonctions reçoivent beaucoup plus de trafic.

Plus de fonctions

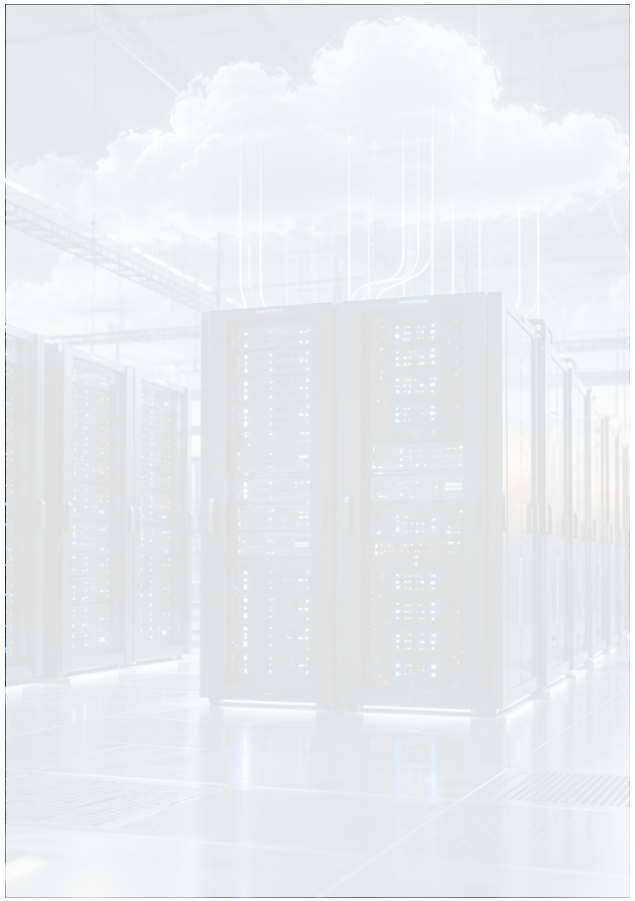
Les modules deviennent nombreux et interdépendants.

Plus de mises à jour

Certaines parties évoluent plus vite que d'autres.

Charges différentes

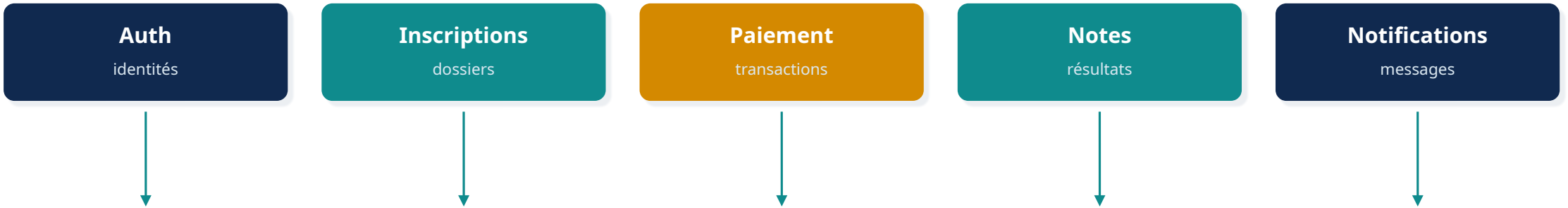
Paiement, notes et notifications n'ont pas les mêmes besoins.



Le découpage doit répondre à un besoin d'évolution, de maintenance ou de montée en charge.

Définition des microservices

Une architecture microservices découpe une application en plusieurs services autonomes.



Caractéristiques essentielles

service autonome
déploiement indépendant
communication par API
supervision et sécurité service par service

Exemple microservices

La plateforme universitaire devient un ensemble de services spécialisés.

Service Authentification

API dédiée • logs • supervision

Service Inscription

API dédiée • logs • supervision

Service Paiement

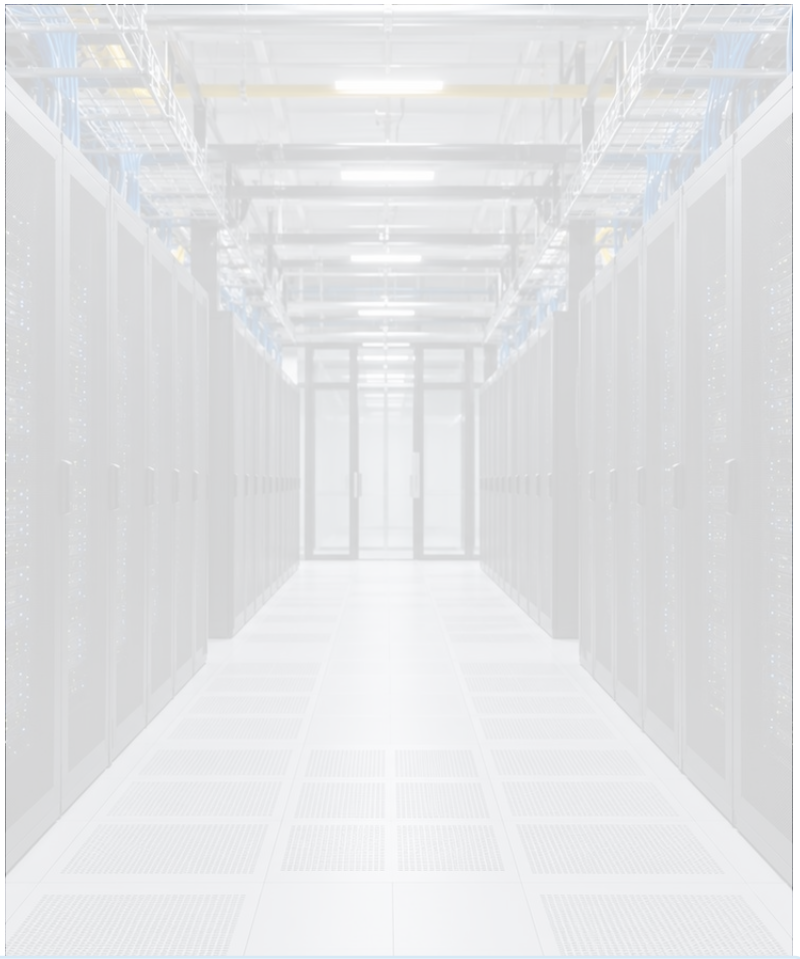
API dédiée • logs • supervision

Service Notes

API dédiée • logs • supervision

Service Notifications

API dédiée • logs • supervision



Chaque service peut évoluer, redémarrer ou être renforcé indépendamment des autres.

Communication entre microservices

Les services échangent généralement via des API et des protocoles réseau.



Protocoles

HTTP/HTTPS
REST
JSON
messagerie éventuelle

Exemple d'appel interservice

```
POST /api/paiements HTTP/1.1
Content-Type: application/json
Authorization: Bearer ...
```

Plus il y a de services, plus la supervision des flux devient importante.

Monolithe vs microservices

Les microservices apportent de la flexibilité, mais augmentent la complexité d'exploitation.

Monolithe	Microservices
Un seul ensemble	Plusieurs services autonomes
Déploiement unique	Déploiements indépendants
Exploitation plus simple	Exploitation plus complexe
Moins de flux réseau	Beaucoup de communications
Montée en charge globale	Montée en charge ciblée

Les microservices ne sont pas automatiquement meilleurs : ils doivent être justifiés par le besoin.

Microservices : impacts infrastructure

Le découpage applicatif augmente les exigences réseau, sécurité et exploitation.

Davantage de services

Point à prévoir dans l'infrastructure.

Davantage de flux réseau

Point à prévoir dans l'infrastructure.

Supervision renforcée

Point à prévoir dans l'infrastructure.

Journalisation centralisée

Point à prévoir dans l'infrastructure.

Sécurité interservices

Point à prévoir dans l'infrastructure.

Load balancing

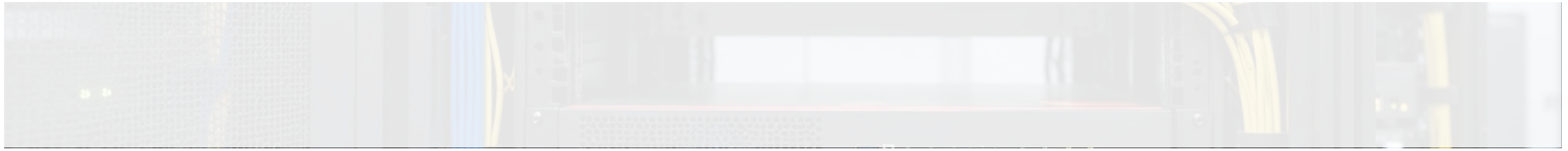
Point à prévoir dans l'infrastructure.

Résilience

Point à prévoir dans l'infrastructure.

Automatisation

Point à prévoir dans l'infrastructure.



Une architecture microservices exige une exploitation mature : monitoring, logs, sécurité, automatisation.

PARTIE 7

API et services Web

Communication entre applications et services

35

Qu’est-ce qu’une API ?

Une API permet à deux applications ou services de communiquer selon des règles définies.



Rôle

Définir les URL, méthodes, paramètres, formats de réponse et règles d’accès.

Exemple

Le portail demande les informations d’un étudiant au service étudiant.

Exemple d’appel API

Une requête API est une requête HTTP structurée.

Requête HTTP

```
GET /api/etudiants/123 HTTP/1.1
Host: portail.universite.sn
Authorization: Bearer eyJhbGciOi...
Accept: application/json
```

Lecture de la requête

GET : consultation de ressource
/api/etudiants/123 : ressource demandée
Authorization : jeton d’accès
JSON : format de réponse attendu



Une API expose des ressources et des opérations accessibles via le réseau.

Exemple de réponse API

Le serveur renvoie des données structurées, souvent au format JSON.

Réponse JSON

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 123,
  "nom": "Diallo",
  "statut": "actif"
}
```

Interprétation

200 OK : demande traitée avec succès
Content-Type : type de données retournées
JSON : format exploitable par l'application
statut : information métier retournée

L'API transforme une demande réseau en traitement applicatif, puis renvoie une réponse structurée.

API REST et services Web

REST utilise les mécanismes HTTP pour exposer des ressources via le réseau.



Des services pour connecter les applications

GET — **Lire une ressource**

Récupère une ressource sans la modifier.

```
GET /api/etudiants/1
```

POST — **Créer une ressource**

Crée une nouvelle ressource sur le serveur.

```
POST /api/etudiants
{ "nom": "Diallo", "prenom": "Awa" }
```

PUT — **Modifier complètement**

Remplace totalement une ressource existante.

```
PUT /api/etudiants/1
{ "nom": "Diallo", "prenom": "Awa",
"niveau": "L2" }
```

PATCH — **Modifier partiellement**

Modifie seulement certains champs d'une ressource.

```
PATCH /api/etudiants/1
{ "niveau": "L3" }
```

DELETE — **Supprimer**

Supprime une ressource sur le serveur.

```
DELETE /api/etudiants/1
```

Services Web

Fonctions accessibles via HTTP/HTTPS par une autre application ou un autre service.

- API REST
- Services SOAP (plus anciens)
- Webhooks
- Services tiers (ex. paiement, SMS, cartes)

Exemple complet (JSON)

Requête — HTTP

```
GET /api/etudiants/1 HTTP/1.1
Host: api.universite.sn
Accept: application/json
Authorization: Bearer <token>
```

Réponse — JSON

```
{
  "id": 1,
  "nom": "Diallo",
  "prenom": "Awa",
  "niveau": "L3",
  "filiere": "Informatique"
}
```

Architecture d'une API REST dans une application web



Bonnes pratiques

- Utiliser HTTPS
- Authentification (JWT, OAuth2)
- Validation des données
- Codes HTTP appropriés
- Gestion des erreurs
- Versionnement des API
- Documentation (Swagger/OpenAPI)

REST, HTTP/HTTPS et JSON forment la base de nombreuses architectures applicatives modernes.

Des API simples, fiables et sécurisées pour des applications connectées.

PARTIE 8 — ARCHITECTURE APPLICATIVE RÉALISTE

Étude de cas finale

Vous êtes l'architecte technique d'une plateforme universitaire.



Une plateforme,
une communauté,
un avenir.

1 Contexte

- 10 000 étudiants
- 500 agents (enseignants, administratifs)
- Accès Internet 24h/24
- Paiement et authentification
- Services en ligne (inscription, résultats, scolarité, etc.)

2 À proposer

- Architecture 2-tiers ou 3-tiers
- Monolithe ou microservices
- Nombre et rôle des serveurs
- Position des API
- Choix des technologies

3 Infrastructure

- Pare-feu
- Load balancer
- Stockage (local / NAS / SAN)
- Réseau et segmentation
- Sauvegarde

4 Exploitation

- Sécurité (authentification, autorisation, chiffrement)
- Haute disponibilité
- Supervision et monitoring
- Journalisation et audit
- Gestion des mises à jour

Utilisateurs

Étudiants
Inscription, cours, résultats, paiement

Agents
Gestion, scolarité, administration

Accès multi-supports (Web / Mobile)

Internet
HTTPS

Sécurité et accès

Pare-feu
Filtrage et sécurité

Load Balancer
Répartition de charge

Couche applicative

Serveurs applicatifs

- Interface Web
- API
- Services métiers
- Microservices ou Monolithe

Couche données

Base de données
PostgreSQL / MySQL (réplication)

Stockage
NAS / SAN

Exploitation

Supervision
Zabbix / Grafana

Sécurité
Logs / SIEM

Sauvegarde
Automatisée

Journalisation
Audit et traçabilité

Objectif

Concevoir une architecture technique fiable, sécurisée, évolutive et adaptée aux besoins d'une plateforme universitaire.

Disponibilité · Scalabilité · Sécurité · Expérience utilisateur

Des solutions pour aujourd'hui, les compétences de demain.